

Large Reasoning Models and Mixture of Experts Models

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology



LMH

Lady Margaret Hall

May 27, 2026

1. Motivation
2. Learning Outcomes
3. What Are Large Reasoning Models (LRMs)?
 1. Leading Large Reasoning Models
 2. Large language models vs. large reasoning models
 3. How Do LRMs Work?
 4. Types of reasoning in LRMs
4. Reasoning vs. Pattern-Matching in LLMs
5. Reasoning Techniques in LLMs
 1. Chain-of-Thought (CoT) Prompting
 2. Tree-of-Thought (ToT) Reasoning
 3. Self-Consistency (SC)

6. Instruction Tuning for Reasoning Tasks
7. Reinforcement Learning for Reasoning Skill Refinement
8. What is a Mixture of Experts (MoE)?
9. Sparse vs. Dense Models: Trade-offs Motivations
10. Gating Mechanisms – How Experts Are Selected
11. Routing in MoE
12. Benefits of MoE
13. Limitations of LRMs and MoE Models
14. References & Further Reading

► Why study LRMs and MoE?

- LLMs like GPT-3 and GPT-4 excel at generating text.
- However, they often rely on **pattern matching**, not true reasoning.
- Real-world applications like **math problem-solving**, **code synthesis**, and **scientific hypothesis generation** require multi-step reasoning.
- Scaling LLMs is costly. **MoE models** allow for massive model sizes with lower computation per inference.

► Examples:

- *GPT-3 can generate realistic text but fails at solving "If Alice has 3 apples and gives Bob 2, how many does she have?" unless prompted with reasoning steps.*
- *DeepMind's AlphaCode uses reasoning and symbolic techniques to solve programming problems—pure LLMs fall short.*

By the end of this session, you should be able to:

- ▶ Define and distinguish Large Reasoning Models (LRMs) from simpler pattern-matching LLMs.
- ▶ Understand advanced reasoning techniques, including Chain-of-Thought (CoT), Tree-of-Thought (ToT), and self-consistency methods.
- ▶ Explain how instruction tuning and reinforcement learning (RL) can refine and enhance reasoning abilities in language models.
- ▶ Grasp the core concepts of Mixture-of-Experts (MoE) architectures, including gating, routing mechanisms, trade-offs, benefits, and limitations.
- ▶ Critically assess future directions and identify open problems in the field of reasoning with language models.

Large Reasoning Models (LRMs)

What Are Large Reasoning Models (LRMs)?

► Definition:

- Large Language Models (LLMs) optimized for **multi-step logical reasoning**
- Emphasize Chain-of-Thought (CoT) style reasoning

► Key Examples:

- OpenAI: o1, o3
- Google: Gemini 2.0 (Flash mode)
- DeepSeek: R1, V3
- QwQ (Alibaba)
- Claude 3.5
- Codestral (Mistral)

► Distinctive Behavior:

- “Thinking internally”
- Producing hidden or explicit reasoning traces
- Step-by-step intermediate computations
- Intermediate reasoning traces (explicit or latent)
- Better performance on symbolic tasks and math
- Often paired with advanced prompting or finetuning

► Examples:

- **PaLM + CoT Prompting:**

Google's PaLM + CoT prompt significantly improved accuracy on GSM8K (grade-school math) from approx 18% to 58%.

- **Gemini 2.0 Flash mode:**

Provides transparent reasoning steps for decision-making.

- **DeepSeek R1:**

Combines expert models for efficient coding and debugging.

Claimed to achieve "Aha!" moments, solving reasoning tasks using structured thinking steps, not just memorization.

Following are some of the leading Large Reasoning Models (LRMs) as of 2024/2025:

► OpenAI O1/O3

- **Architecture:** Dense Transformer, RLHF-optimized.
- **Training Data Transparency:** Proprietary, limited details.
- **Strengths:** High-quality reasoning, STEM, coding.
- **Reasoning Approach:** Adjustable effort (Low/Med/High), step-by-step.
- **Efficiency:** Adjustable reasoning mode, 200K context window.
- **Best Use Case:** General AI, STEM research, API function calling.

► DeepSeek R1

- **Architecture:** Mixture of Experts (MoE, 671B total, 37B active).
- **Training Data Transparency:** Somewhat open (14.8T tokens).
- **Strengths:** Expert coding & debugging, efficient MoE.
- **Reasoning Approach:** Multi-expert collaboration, self-improvement.
- **Efficiency:** High token efficiency (MoE, 312 tokens/sec).
- **Best Use Case:** Enterprise coding assistant, code debugging.

► Gemini 2.0 (Flash Thinking)

- **Architecture:** Multimodal Transformer (Text+Image+Speech).
- **Training Data Transparency:** Proprietary, multimodal (text, images, APIs).
- **Strengths:** Multimodal (images, text, speech), deep reasoning.
- **Reasoning Approach:** Flash Thinking mode (transparent thought process).
- **Efficiency:** 1M token context (experimental), optimized for speed.
- **Best Use Case:** AI assistant (multimodal, research, planning).

► Claude 3.5

- **Architecture:** Dense Transformer, fine-tuned for dialogue & ethics.
- **Training Data Transparency:** Partially open (some RLHF data available).
- **Strengths:** Long-form dialogue, ethical AI, context memory.
- **Reasoning Approach:** Contextual understanding, ethical guardrails.
- **Efficiency:** Long context (200K), RLHF for alignment.
- **Best Use Case:** Long-form assistant, decision-making support.

► Codestral (Mistral)

- **Architecture:** Dense Transformer, code-focused.
- **Training Data Transparency:** Proprietary, open benchmarks used.
- **Strengths:** Cost-effective code generation, small but optimized.
- **Reasoning Approach:** Optimized for coding efficiency & debugging.
- **Efficiency:** High speed (small model, transformer optimized).
- **Best Use Case:** Cost-effective coding LLM.

► QwQ (Alibaba)

- **Architecture:** Dense Transformer, fine-tuned for reasoning.
- **Training Data Transparency:** Limited transparency, strong math focus.
- **Strengths:** Math, logical puzzles, chain-of-thought self-verification.
- **Reasoning Approach:** Self-reflection, chain-of-thought.
- **Efficiency:** Small model (32B), inference-efficient.
- **Best Use Case:** Mathematical proofs, advanced reasoning.

Large language models (LLMs) and large reasoning models (LRMs) share foundational technologies but serve different purpose:

► Core Function:

- **LLMs:** Learn patterns in data to generate fluent, human-like text.
- **LRMs:** Extend LLMs to solve problems requiring logical reasoning and contextual understanding.

► Use Cases:

- **LLMs:** Content generation, language translation, summarization, and answering user queries.
- **LRMs:** Math problem solving, interpreting ambiguous clinical data, and complex decision-making.

► Reasoning Ability:

- **LLMs:** Limited structured reasoning; may struggle with multi-step logic or ambiguity.
- **LRMs:** Trained to apply consistent reasoning steps; outputs are more logical and verifiable.

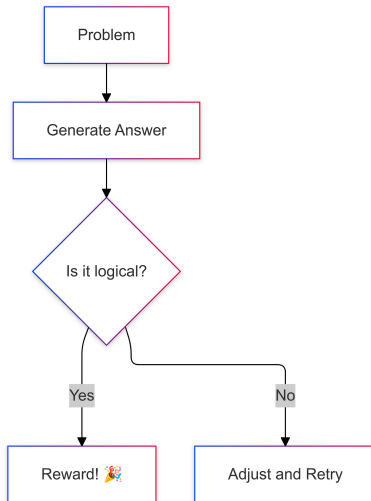
► Performance:

- **LLMs:** Fast response times; optimized for scalability and speed.
- **LRMs:** Slower response times; require more processing to reason through problems step by step.

► Best Suited For:

- **LLMs:** High-volume, low-risk tasks where speed is critical.
- **LRMs:** High-stakes or complex tasks where accuracy, explainability, and logic matter.

How Do LRMs Work?



LRMs use a combination of training methods and prompt strategies to enhance the reasoning capabilities of LLMs.

► Training on Enriched Datasets

- Includes reasoning-focused examples
- Real-world scenarios with clear outcomes
- Teaches both correct answers and reasoning steps

► Reinforcement Learning (RL)

- Rewards for logical, correct answers
- Penalties for incorrect or inconsistent reasoning
- Reinforces desirable reasoning patterns

► Human Feedback (RLHF)

- Human reviewers guide model outputs

- Refines nuanced reasoning strategies
- Useful in domains needing expertise (e.g., healthcare, finance)

► Prompt Engineering

- Carefully designed prompts activate reasoning
- Guides model through multi-step or layered tasks
- Generic prompts may not trigger deep reasoning

► Chain-of-Thought (CoT) Prompting

- Breaks problems into smaller steps
- Explores multiple reasoning paths
- Improves transparency and accuracy
- Essential for auditability and explainability

Large reasoning models (LRMs) employ several types of reasoning to simulate or support human-like problem solving:

► Deductive Reasoning

- Applies general rules to specific cases for logically certain conclusions (top-down reasoning).
- Excels at structured, rule-based tasks requiring high logical consistency.
- Example domains: regulatory compliance, medical protocol analysis.
- *Example:*
Rule: All patients with a fever and cough should be tested for influenza.
Case: Patient A has a fever and cough.
Deduction: Patient A should be tested for influenza.

► Inductive Reasoning

- Draws general conclusions from specific observations in data.
- Supports probabilistic predictions and generalization to unseen inputs.
- Useful in dynamic, data-rich scenarios (e.g., fraud detection).
- *Example:*
Observation: 90% of emails with suspicious links are phishing attempts.
Induction: An email with a suspicious link is likely to be a phishing attempt.

► Abductive Reasoning

- Infers the most likely explanation from incomplete or uncertain evidence.
- Generates hypotheses rather than definitive answers.
- Effective in domains like medical diagnostics, where plausible interpretations are needed.
- *Example:*
Evidence: The lawn is wet in the morning.
Possible explanations: It rained overnight, or the sprinkler was on.
Abduction: If the weather report says no rain, the most likely explanation is the sprinkler was on.

► Analogical Reasoning

- Identifies similarities between different situations or datasets.
- Transfers insights from one context to another for novel problem solving.
- Example: recommending products based on similar customer behavior.
- *Example:*
Situation A: A student learns to solve quadratic equations by factoring.
Situation B: The student applies the same factoring method to solve cubic equations.
Analogy: The approach used for quadratics is adapted to cubics.



Reasoning vs. Pattern-Matching in LLMs

► Pattern-Matching Models

- **How it works:** LLMs are trained on vast amount of text data to learn statistical associations between words and phrases.
- **Characteristics:**
 - Generate text based on learned patterns without deeper understanding.
 - Often produce fluent and coherent text.
 - May struggle with complex reasoning tasks or multi-step logic.
- **Example:** Given the prompt “The capital of France is”, the model outputs “Paris” by recalling frequent patterns in the training data.
- **Limitations:**
 - May produce plausible-sounding but incorrect answers if the pattern is misleading.
 - Struggles with tasks that require connecting multiple facts or reasoning through steps.

► Reasoning-Focused Models

- **How it works:** LRMs extend LLMs by incorporating structured reasoning techniques.
- **Characteristics:**
 - Generate structured intermediate steps to arrive at conclusions.
 - Use multi-hop inference to connect information across several statements or facts.
 - Can break down complex problems into smaller, logical steps.
- **Example:** To answer “If Alice has 3 apples and buys 2 more, how many does she have?”, the model first adds 3 and 2, then states the answer.
- **Benefits:**
 - Provides explanations or shows their thought process, making answers more transparent.
 - Better at tasks that require logic, calculation, or combining information from different sources.

► Examples

- “Which is heavier: a kilogram of feathers or a kilogram of iron?”

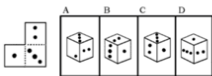
A pattern-matching model may be misled by the association of “iron” with heaviness.

A reasoning model will recognize that both weigh the same.

- In logic puzzles, pattern-based models tend to contradict earlier premises

Reasoning-aware models (e.g., using Chain-of-Thought prompting) remain consistent and logical.

Still Terrible



Box folding: F
LLMs cannot solve box-folding problems and struggle with abstract figures and 3d spaces.



Abstract Map Navigation: D
LLMs cannot solve complex problems like route-planning in abstract 2d spaces.



Venn diagram fail



Supposed to be an unfolded box

Drawing diagrams: F
All tested models are comically bad at producing diagrams to exact, or even loose specifications.

Improving



Stacking objects: B+
Claude solved a puzzle GPT failed on a year ago. Progress!



Real Map Navigation: C+
Models showed uneven but improving performance in multi-stop navigation.



Spatial reasoning in photos: B
Models seem to do better reasoning from photos of real scenes than from abstractions.

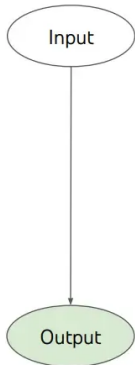


Venn Diagrams: B
Claude and GPT can reason from verbal descriptions of Venn diagrams.

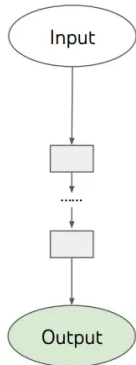
► Empirical Separation: When Does Reasoning Matter?

- On simple factual questions, both model types may perform well.
- For challenging tasks like GSM8K (grade-school math), symbolic reasoning, or logic puzzles, pattern-matching alone is not enough.
- Success on these benchmarks improves dramatically when models are prompted to reason step by step (e.g., using Chain-of-Thought prompting).
- *Example:* In multi-step math problems, models that show their work (reasoning-focused) achieve much higher accuracy than those that just guess the answer.
- This highlights the need for explicit reasoning capabilities in advanced language models.

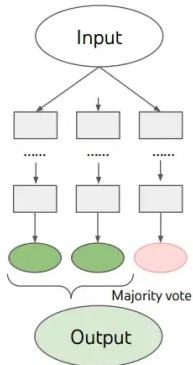
Reasoning Techniques in LLMs



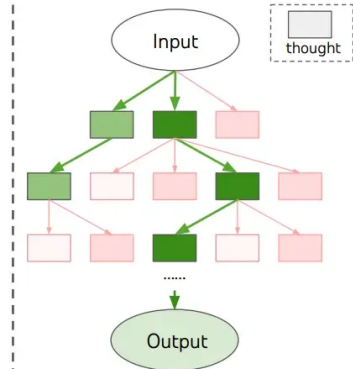
(a) Input-Output Prompting (IO)



(c) Chain of Thought Prompting (CoT)



(c) Self Consistency with CoT (CoT-SC)



(d) Tree of Thoughts (ToT)

► What is CoT?

- A prompting technique that encourages LLMs to generate intermediate reasoning steps before arriving at a final answer.
- Helps models to break down complex problems into manageable parts.

► How it works:

- Instead of asking for a direct answer, the prompt includes examples that show the reasoning process.
- The model is guided to think step-by-step, mimicking human-like reasoning.

► Benefits:

- Improves accuracy on tasks requiring multi-step reasoning, such as math problems or logical puzzles.
- Makes the model's reasoning process more transparent, allowing users to follow its logic.

► Example:

- Instead of asking "What is 24 divided by 6?", the prompt might be "To find 24 divided by 6, we can think: 6 goes into 24 how many times? Let's break it down step by step."

► What is Few-shot CoT?

- A variant of CoT prompting that provides a few examples of reasoning steps before the actual question.
- Helps the model learn the reasoning pattern from the examples.

► How it works:

- The prompt includes several examples of problems and their step-by-step solutions.
- The model uses these examples to understand how to approach the new problem.

► Benefits:

- Enhances the model's ability to generalize from examples, improving performance on similar tasks.
- Reduces the need for extensive training on specific reasoning tasks.

► Example:

- The prompt might include examples like "If 6 times 4 equals 24, then what is 6 times 5?" followed by the reasoning steps leading to the answer.
- The model learns to apply similar reasoning to new problems presented in the same format.

► What is Zero-shot CoT?

- A variant of CoT prompting where the model is prompted to reason through a problem without any prior examples.
- The model generates its own reasoning steps based on the problem statement.

► How it works:

- The prompt directly asks the model to think through the problem step-by-step, without providing specific examples.
- The model relies on its training to generate a logical sequence of reasoning.

► Benefits:

- Useful for novel problems where no examples are available.
- Encourages the model to apply its learned reasoning skills flexibly.

► Example:

- The prompt might be "To solve the problem of how many apples Alice has after giving some to Bob, think step-by-step about what happens when she gives away apples."
- The model generates its own reasoning steps based on the prompt.

Zero-shot CoT Prompting (cont.)

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Comparison of Few-shot and Zero-shot CoT Prompting. Source: Zhang et al. (2022)

AUTOMATIC CHAIN OF THOUGHT PROMPTING IN LARGE LANGUAGE MODELS

Zhuosheng Zhang^{†,*}, Aston Zhang[‡], Mu Li[‡], Alex Smola[‡]

[†]Shanghai Jiao Tong University, [‡]Amazon Web Services

ABSTRACT

Large language models (LLMs) can perform complex reasoning by generating intermediate reasoning steps. Providing these steps for prompting demonstrations is called chain-of-thought (CoT) prompting. CoT prompting has two major paradigms. One leverages a simple prompt like “Let’s think step by step” to facilitate step-by-step thinking before answering a question. The other uses a few manual demonstrations one by one, each composed of a question and a reasoning chain that leads to an answer. The superior performance of the second paradigm hinges on the hand-crafting of task-specific demonstrations one by one. We show that such manual efforts may be eliminated by leveraging LLMs with the “Let’s think step by step” prompt to generate reasoning chains for demonstrations one by one, i.e., *let’s think not just step by step, but also one by one*. However, these generated chains often come with mistakes. To mitigate the effect of such mistakes, we find that diversity matters for automatically constructing demonstrations. We propose an automatic CoT prompting method: Auto-CoT. It samples questions with diversity and generates reasoning chains to construct demonstrations. On ten public benchmark reasoning tasks with GPT-3, Auto-CoT consistently matches or exceeds the performance of the CoT paradigm that requires manual designs of demonstrations. Code is available at <https://github.com/amazon-research/auto-cot>

► What is Auto-CoT?

- An automated approach to generating Chain-of-Thought prompts.
- Uses algorithms to create reasoning steps based on the problem context.

► How it works:

- The system analyzes the problem and automatically generates a sequence of reasoning steps.
- This can be done using techniques like reinforcement learning or heuristic methods.

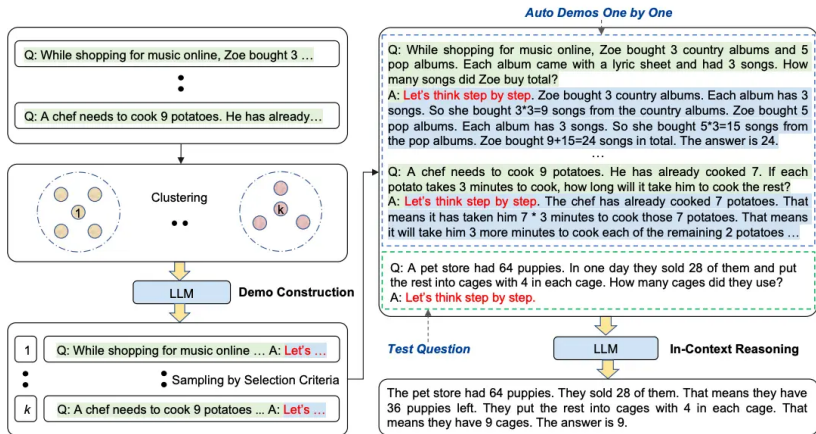
► Benefits:

- Reduces the manual effort required to create effective CoT prompts.
- Can adapt to a wide range of problems without needing specific examples.

► Example:

- The system might take a problem like "If Alice has 3 apples and gives 2 to Bob, how many does she have left?" and automatically generate reasoning steps like "Start with 3 apples, subtract 2 apples given to Bob, and conclude with the remaining number of apples."

Auto-CoT Prompting (cont.)



Auto-CoT Prompting. Source: Zhang et al. (2022)

Algorithm 1 Cluster

Require: A set of questions $\mathcal{Q}$ and the number of demonstrations k

Ensure: Sorted questions $\mathbf{q}^{(i)} = [q_1^{(i)}, q_2^{(i)}, \dots]$ for each cluster i ($i = 1, \dots, k$)

- 1: **procedure** CLUSTER($\mathcal{Q}, k$)
 - 2: **for** each question q in $\mathcal{Q}$ **do**
 - 3: Encode q by Sentence-BERT
 - 4: Cluster all the encoded question representations into k clusters
 - 5: **for** each cluster $i = 1, \dots, k$ **do**
 - 6: Sort questions $\mathbf{q}^{(i)} = [q_1^{(i)}, q_2^{(i)}, \dots]$ in the ascending order of the distance to the cluster center
 - 7: **return** $\mathbf{q}^{(i)}$ ($i = 1, \dots, k$)
-

Algorithm 2 Construct

Require: Sorted questions $\mathbf{q}^{(i)} = [q_1^{(i)}, q_2^{(i)}, \dots]$ for each cluster i ($i = 1, \dots, k$), empty demonstration list $\mathbf{d}$

Ensure: Demonstration list $\mathbf{d} = [d^{(1)}, \dots, d^{(k)}]$

```
1: procedure CONSTRUCT( $\mathbf{q}^{(1)}, \dots, \mathbf{q}^{(k)}$ )
2:   for each cluster  $i = 1, \dots, k$  do
3:     for each question  $q_j^{(i)}$  in  $\mathbf{q}^{(i)}$  do
4:       Generate rationale  $r_j^{(i)}$  and answer  $a_j^{(i)}$  for  $q_j^{(i)}$  using
         Zero-Shot-CoT
5:       if  $q_j^{(i)}, r_j^{(i)}$  satisfy selection criteria then
6:         Add  $d^{(i)} = [\text{Q: } q_j^{(i)}, \text{A: } r_j^{(i)} \circ a_j^{(i)}]$  to  $\mathbf{d}$ 
7:       break
8:   return  $\mathbf{d}$ 
```

Tree of Thoughts: Deliberate Problem Solving with Large Language Models

Shunyu Yao
Princeton University

Dian Yu
Google DeepMind

Jeffrey Zhao
Google DeepMind

Izhak Shafran
Google DeepMind

Thomas L. Griffiths
Princeton University

Yuan Cao
Google DeepMind

Karthik Narasimhan
Princeton University

Abstract

Language models are increasingly being deployed for general problem solving across a wide range of tasks, but are still confined to token-level, left-to-right decision-making processes during inference. This means they can fall short in tasks that require exploration, strategic lookahead, or where initial decisions play a pivotal role. To surmount these challenges, we introduce a new framework for language model inference, “Tree of Thoughts” (ToT), which generalizes over the popular “Chain of Thought” approach to prompting language models, and enables exploration over coherent units of text (“thoughts”) that serve as intermediate steps toward problem solving. ToT allows LMs to perform deliberate decision making by considering multiple different reasoning paths and self-evaluating choices to decide the next course of action, as well as looking ahead or backtracking when necessary to make global choices. Our experiments show that ToT significantly enhances language models’ problem-solving abilities on three novel tasks requiring non-trivial planning or search: Game of 24, Creative Writing, and Mini Crosswords. For instance, in Game of 24, while GPT-4 with chain-of-thought prompting only solved 4% of tasks, our method achieved a success rate of 74%. Code repo with all prompts: <https://github.com/princeton-nlp/tree-of-thought-llm>.

► What is ToT?

- A reasoning technique that structures the thought process as a tree, allowing for exploration of multiple reasoning paths.
- Each node in the tree represents a step in the reasoning process, and branches represent different possible paths or decisions.

► How it works:

- The model generates a tree structure where each branch represents a different reasoning path.
- The model can explore multiple paths simultaneously, evaluating the outcomes of each.

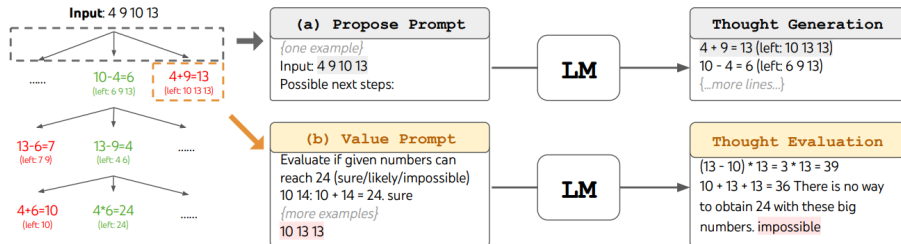
► Benefits:

- Allows for more complex reasoning by considering multiple possibilities at once.
- Helps the model avoid getting stuck in a single line of reasoning, improving problem-solving capabilities.

► Example:

- For a problem like "If Alice has 3 apples and gives some to Bob, how many does she have left?", the model might create a tree with branches for different amounts given to Bob (e.g., 1, 2, or 3 apples) and explore the outcomes of each branch.

Tree-of-Thought (ToT) Reasoning (cont.)



ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation. Source: [Yao et al. \(2023\)](#)

Algorithm 1 ToT-BFS($x, p_\theta, G, k, V, T, b$)

Require: Input x , LM p_θ , thought generator $G()$ & size limit k , states evaluator $V()$, step limit T , breadth limit b .

$S_0 \leftarrow \{x\}$

for $t = 1, \dots, T$ **do**

$S'_t \leftarrow \{[s, z] \mid s \in S_{t-1}, z_t \in G(p_\theta, s, k)\}$

$V_t \leftarrow V(p_\theta, S'_t)$

$S_t \leftarrow \arg \max_{S \subset S'_t, |S|=b} \sum_{s \in S} V_t(s)$

end for

return $G(p_\theta, \arg \max_{s \in S_T} V_T(s), 1)$

Algorithm 2 ToT-DFS($s, t, p_\theta, G, k, V, T, v_{th}$)

Require: Current state s , step t , LM p_θ , thought generator $G()$ and size limit k , states evaluator $V()$, step limit T , threshold v_{th}

if $t > T$ **then** record output $G(p_\theta, s, 1)$

end if

for $s' \in G(p_\theta, s, k)$ **do** $\triangleright$ sorted candidates

if $V(p_\theta, \{s'\})(s) > v_{thres}$ **then** $\triangleright$ pruning

 DFS($s', t + 1$)

end if

end for

Tree-of-Thought (ToT) Reasoning (cont.)

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC ($k=100$)	9.0%
ToT (ours) ($b=1$)	45%
ToT (ours) ($b=5$)	74%
IO + Refine ($k=10$)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

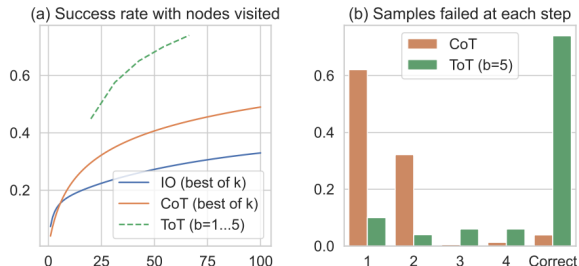


Figure 3: Game of 24 (a) scale analysis & (b) error analysis.

Tree-of-Thought (ToT) Reasoning (cont.)

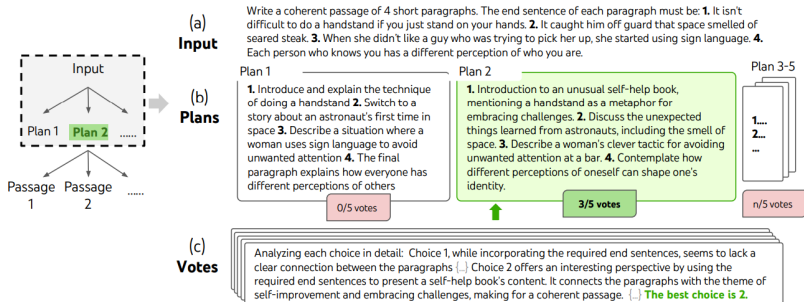


Figure 4: A step of deliberate search in a randomly picked Creative Writing task. Given the input, the LM samples 5 different plans, then votes 5 times to decide which plan is best. The majority choice is used to consequently write the output passage with the same sample-vote procedure.

Tree-of-Thought (ToT) Reasoning (cont.)

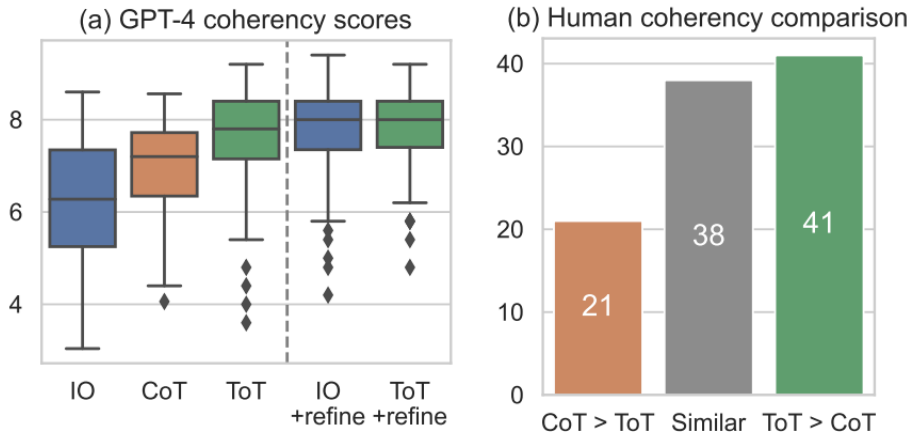


Figure 5: Creative Writing results.

SELF-CONSISTENCY IMPROVES CHAIN OF THOUGHT REASONING IN LANGUAGE MODELS

Xuezhi Wang^{†‡} Jason Wei[†] Dale Schuurmans[†] Quoc Le[†] Ed H. Chi[†]

Sharan Narang[†] Aakanksha Chowdhery[†] Denny Zhou^{†§}

[†]Google Research, Brain Team

[‡]xuezhw@google.com, [§]dennyzhou@google.com

ABSTRACT

Chain-of-thought prompting combined with pre-trained large language models has achieved encouraging results on complex reasoning tasks. In this paper, we propose a new decoding strategy, *self-consistency*, to replace the naive greedy decoding used in chain-of-thought prompting. It first samples a diverse set of reasoning paths instead of only taking the greedy one, and then selects the most consistent answer by marginalizing out the sampled reasoning paths. Self-consistency leverages the intuition that a complex reasoning problem typically admits multiple different ways of thinking leading to its unique correct answer. Our extensive empirical evaluation shows that self-consistency boosts the performance of chain-of-thought prompting with a striking margin on a range of popular arithmetic and commonsense reasoning benchmarks, including GSM8K (+17.9%), SVAMP (+11.0%), AQuA (+12.2%), StrategyQA (+6.4%) and ARC-challenge (+3.9%).

► What is Self-Consistency?

- A technique that improves the reliability of LLM outputs by generating multiple responses to the same prompt and selecting the most consistent answer.
- Helps mitigate the variability in LLM responses, especially in tasks requiring reasoning or complex decision-making.

► How it works:

- The model generates several responses to a given prompt.
- These responses are then evaluated for consistency, and the most frequent or coherent response is selected as the final answer.

► Benefits:

- Reduces the impact of noise and randomness in LLM outputs, leading to more reliable results.
- Enhances the model's performance on tasks that require stable and consistent reasoning.

► Example:

- For a question like "What is the capital of France?", the model might generate multiple responses such as "Paris", "Paris is the capital", and "The capital city is Paris". The most consistent response ("Paris") would be selected as the final answer.

Self-Consistency (SC) (cont.)

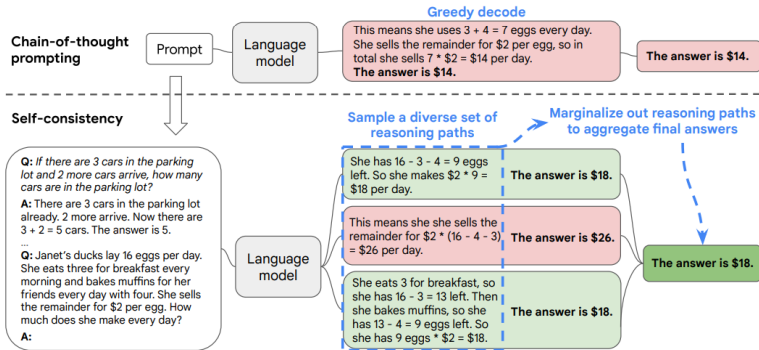


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the “greedy decode” in CoT prompting by sampling from the language model’s decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

Source: Wang et al. (2022)

Self-Consistency (SC) (cont.)

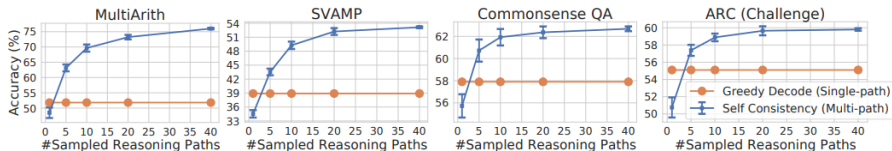


Figure 2: Self-consistency (blue) significantly improves accuracy over CoT-prompting with greedy decoding (orange) across arithmetic and commonsense reasoning tasks, over LaMDA-137B. Sampling a higher number of diverse reasoning paths consistently improves reasoning accuracy.

	GSM8K	StrategyQA
Question	<i>Henry made two stops during his 60-mile bike trip. He first stopped after 20 miles. His second stop was 15 miles before the end of the trip. How many miles did he travel between his first and second stops?</i>	<i>Yes or no: Is Albany, Georgia the most populous US Albany?</i>
Greedy Decode	He traveled 60 miles in total. He stopped after 20 miles, so he traveled 60 - 20 = 40 miles between the first and second stops. The answer is 40.	Albany, Georgia is the most populous US Albany. Thus, the answer is yes.
Sampled Path 1	Henry travelled a total of 60 miles. His first stop was 20 miles in, and his second stop was 15 miles before the end. So between his first and second stops he travelled 60 - 20 - 15 = 25 miles. The answer is 25.	The most populous US Albany is Albany, New York. Thus, Albany, Georgia is not the most populous US Albany. So the answer is no.
Sampled Path 2	He made two stops during a 60-mile trip. The first was 20 miles into the trip. The second was 15 miles before the end of the trip. This means the second stop was 60 - 15 = 45 miles into the trip. Since he made the stops in order, the second stop must have been 45 - 20 = 25 miles after the first stop. The answer is 25.	Albany, Georgia has a population of about 88,000. Albany, New York has a population of about 95,000. Thus, Albany, Georgia is not the most populous US Albany. So the answer is no.

Table 4: Examples where self-consistency helps repair the errors over greedy decode, on PaLM-540B. Two sampled reasoning paths that are consistent with the ground truth are shown.

Self-Consistency (SC) (cont.)

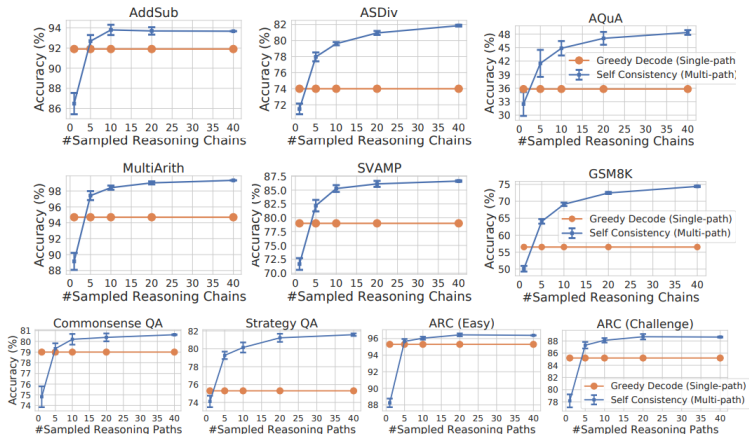
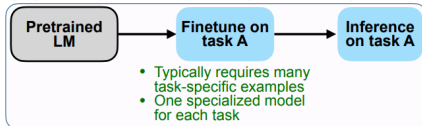


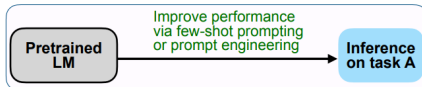
Figure 8: Self-consistency (blue) significantly improves accuracy across various arithmetic and commonsense reasoning tasks, over PaLM-540B. Sampling a higher number of diverse reasoning paths consistently helps reasoning accuracy.

Instruction Tuning for Reasoning Tasks

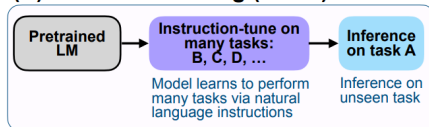
(A) Pretrain–finetune (BERT, T5)



(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



Instruction Tuning for Large Language Models: A Survey

Shengyu Zhang[♦], Linfeng Dong[♦], Xiaoya Li[♦], Sen Zhang[♦]

Xiaofei Sun[♦], Shuhe Wang[♦], Jiwei Li^{♦♦}, Runyi Hu[♦]

Tianwei Zhang[▲], Fei Wu[♦] and Guoyin Wang[♦]

Abstract

This paper surveys research works in the quickly advancing field of instruction tuning (IT), a crucial technique to enhance the capabilities and controllability of large language models (LLMs). Instruction tuning refers to the process of further training LLMs on a dataset consisting of (INSTRUCTION, OUTPUT) pairs in a supervised fashion, which bridges the gap between the next-word prediction objective of LLMs and the users' objective of having LLMs adhere to human instructions. In this work, we make a systematic review of the literature, including the general methodology of IT, the construction of IT datasets, the training of IT models, and applications to different modalities, domains and application, along with analysis on aspects that influence the outcome of IT (e.g., generation of instruction outputs, size of the instruction dataset, etc). We also review the potential pitfalls of IT along with criticism against it, along with efforts pointing out current deficiencies of existing strategies and suggest some avenues for fruitful research. [†](#) [†](#) [†](#) [†](#)

► What is Instruction Tuning?

- Fine-tuning LLMs on a dataset of instruction-response pairs.
- Enhances the model's ability to follow instructions and perform tasks.

► Why is it important?

- Improves model performance on diverse tasks.
- Enables better generalization to unseen tasks.

Why Instruction Tune LLMs?

- ▶ Pre-trained LLMs are optimized for next-word prediction, not for following instructions or engaging in conversations.
- ▶ Without instruction tuning, LLMs may generate text that is linguistically correct but not aligned with user intent.
- ▶ Instruction tuning uses supervised learning on instruction-response pairs, teaching the model to generate helpful and relevant outputs for user prompts.
- ▶ Fine-tuning is much more efficient than training a large model from scratch and can be performed with less data and compute.
- ▶ Instruction tuning bridges the gap between generic language modeling and practical, task-oriented use cases, making LLMs more useful and predictable.

Summary of Instruction Tuning

- ▶ **Definition:** Fine-tuning LLMs on diverse instruction-response pairs.
- ▶ **Key Elements:**
 - *Instruction:* Task description in natural language.
 - *Additional Information:* Optional context or input.
 - *Desired Output:* Target response for evaluation.
- ▶ **Benefits:**
 - Improves ability to follow instructions.
 - Reduces need for prompt engineering and few-shot examples.
 - Enhances generalization to unseen tasks.
- ▶ **Applications:**
 - Translation, question answering, reading comprehension, NLI.
 - Broad improvements across reasoning and practical tasks.

Examples of Instruction Tuning in Practice

► FLAN-T5:

- Instruction-tuned T5 models show strong improvements on reasoning benchmarks such as DROP (reading comprehension), ARC (science questions), and MultiRC (multi-sentence reasoning).
- For example, when given instructions like “Answer the following science question” or “Read the passage and answer the question,” FLAN-T5 produces more accurate and relevant responses.
- *Example:*
 - **Instruction:** “Read the passage and answer: What did the boy do after school?”
 - **Passage:** “The boy finished his homework and went to play soccer.”
 - **FLAN-T5 Output:** “He went to play soccer.”

► FLAN-MoE:

- Mixture-of-Experts (MoE) models, when instruction-tuned, outperform larger dense models like PaLM on reasoning tasks, while using less computation.
- FLAN-MoE can handle a wide range of instructions, such as translation, summarization, and logical reasoning, with high accuracy.
- *Example:*
 - **Instruction:** “Translate this sentence to French: The cat is sleeping.”
 - **FLAN-MoE Output:** “Le chat dort.”

Why Does Instruction Tuning Work?

- ▶ Models learn to **follow instructions step by step**, rather than just predicting the next word or memorizing answers.
- ▶ Encourages **logical reasoning**: Models are guided to explain their answers or show their work, which helps in tasks like math, logic puzzles, and multi-step questions.
- ▶ **Generalization**: Instruction-tuned models can handle new types of questions or tasks they have not seen before, simply by following the provided instruction.
- ▶ **Variety of Examples**:
 - *Math*: "Solve: What is 15 divided by 3?" → "5"
 - *Reasoning*: "Is a whale a mammal or a fish? Explain your answer." → "A whale is a mammal because it gives birth to live young and breathes air."
 - *Summarization*: "Summarize the following paragraph in one sentence."

Instruction Tuning vs. Multi-Task Fine-Tuning (FLAN Ablation Study)

- ▶ **Research Question:** Are the benefits of instruction tuning due to the instructions themselves, or just from fine-tuning on multiple tasks?
- ▶ **Ablation Setups:**
 - *No Template:* Only input-output pairs, e.g., input: “the dog runs”, output: “le chien court”.
 - *Dataset Name:* Input prepended with task and dataset, e.g., “[Translation: WMT 14 to French] The dog runs”.
 - *FLAN Instructions:* Full natural language instruction, e.g., “Please translate this sentence to French: ‘The dog runs.’ ”
- ▶ **Results:**
 - Instruction-tuned models achieved over 18% higher accuracy than “no template” and over 8% higher than “dataset name” on zero-shot tasks.
 - **Conclusion:** Explicit instructions are crucial for improving zero-shot generalization, not just multi-task fine-tuning.

Chain-of-Thought (CoT) Fine-Tuning

- ▶ **How is it done?** This can be achieved by:
 - Few-shot prompting with exemplars that demonstrate sequential reasoning.
 - Appending phrases like “think step by step” to prompts.
- ▶ **Why is it important?**
 - CoT prompting significantly enhances zero-shot performance on arithmetic, symbolic, and logical reasoning tasks.
 - Research (Wei et al.) shows that instruction tuning without CoT tasks degrades performance on CoT evaluations.
 - Including CoT tasks in instruction tuning improves performance across all evaluations.
- ▶ **Key Insight:** Fine-tuning on CoT tasks—both with and without few-shot exemplars—enables models to better perform step-by-step reasoning, rather than jumping to linguistically plausible answers.

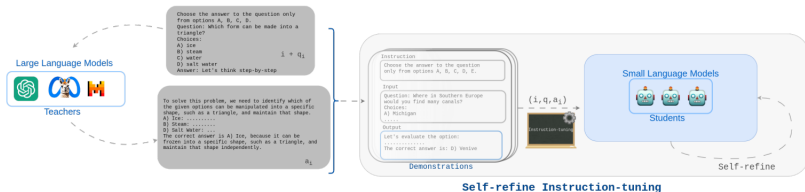


Figure 1: In Self-refine Instruction-tuning, the Demonstrations delivered by teacher models are used to align reasoning abilities in a teacher-student setting. Following the transference of step-wise reasoning knowledge via instruction tuning, the students Self-refine their abilities with the support of Direct Preference Optimization methods.

Instruction tuning process. Source: [Ranaldi et al. \(2024\)](#)

Instruction Tuning (cont.)

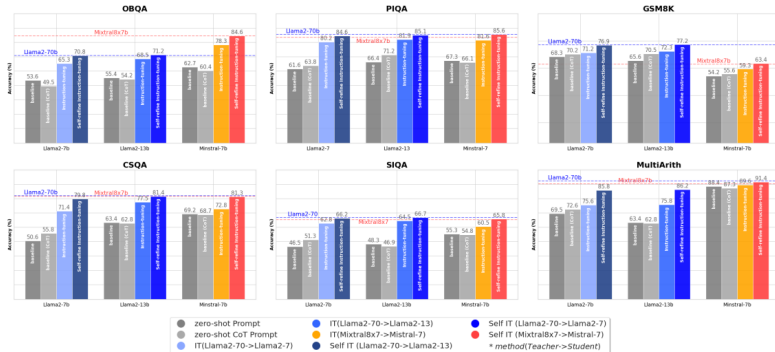


Figure 2: Accuracies (%) on benchmarks (Section 3.1) before Instruction-tuning (i.e., Baselines and Baseline CoT), after Instruction-tuning (IT) performed on Demonstrations delivering CoT and finally behind the Self-refine Instruction-tuning phase (Self IT). In particular, the models were instructed via Demonstrations delivered by in-family LLMs (as described in the legend, we use the notation *method(Teacher->Student)*).

Source: [Ranaldi et al. \(2024\)](#)

Prompt for task: OBQA, CSQA, PIQA, SIQA

Choose the answer to the question only
from options A, B, C, [...].

Question: <Question>

Choices:

A) <Option1>

B) <Option2>

C) <Option3>

....

Answer:

Prompt for task: GSM8k, MultiArith

Answer the following mathematical
question with numerical solution.

Question: <Question>

Answer:

Table 2: Example of input-prompt for multiple-choices (left) and mathematical (right) question-answering benchmarks.

Prompt for task: OBQA, CSQA, PIQA, SIQA

Choose the answer to the question only
from options A, B, C, [...].

Question: <Question>

Choices:

A) <Option1>

B) <Option2>

C) <Option3>

....

Answer: Let's think step by step

Prompt for task: GSM8k, MultiArith

Answer the following mathematical
question with numerical solution.

Question: <Question>

Answer: Let's think step by step

Table 3: Example **Zero-shot CoT** of input-prompt for multiple-choices (left) and mathematical (right) question-answering benchmarks (approach used in this work).

Instruction

Choose the answer to the question only from options A, B, C, D.

Input

Question: Which form can be made into a triangle?

Choices:

- A) ice
- B) steam
- C) water
- D) salt water

Answer: **Let's think step-by-step**

Output (that is the Answer produced by teacher model)

Answer:

A) ice: Ice is a solid form of water. In solid form, substances can be molded or shaped into various forms, including triangles. If you had a block of ice, you could carve or mold it into a triangular shape.

B) steam: Steam is the gaseous form of water. Being a gas, it doesn't have a fixed shape. You can't shape steam into a triangle without containing it within some form of solid boundary, and even then, the steam itself isn't triangular - the container is.

C) water: Water is a liquid that takes the shape of its container. You can pour water into a triangular container, and while the volume inside would be triangular, the water itself isn't inherently triangular.

D) salt water: This is essentially the same as option C, but with salt dissolved in it. Being a liquid, it takes the shape of its container.

Given these considerations, the only substance among the options that can be shaped into a triangle by itself is **A) Ice**.

Table 6: Example of Demonstration produced by the GPT-3.5 answer on an Open Book Question Answering benchmark instance. The structure is composed by: **Instruction**, **Input** and **Output**.

Source: **Ranaldi et al. (2024)**

Challenges and Limitations of Instruction Tuning

► Dataset Quality and Diversity:

- Large, high-quality datasets are costly to create.
- Reliance on few open-source datasets may reduce diversity.

► Bias Propagation:

- Proprietary LLMs used for instruction generation can spread their biases to open-source models.

► Evaluation Concerns:

- Evaluation often uses proprietary models, which may not reveal all biases or limitations.

► Superficial Learning:

- Smaller models may mimic style without true reasoning.
- Gains may reflect pattern matching, not understanding.

► Overfitting to Training Tasks:

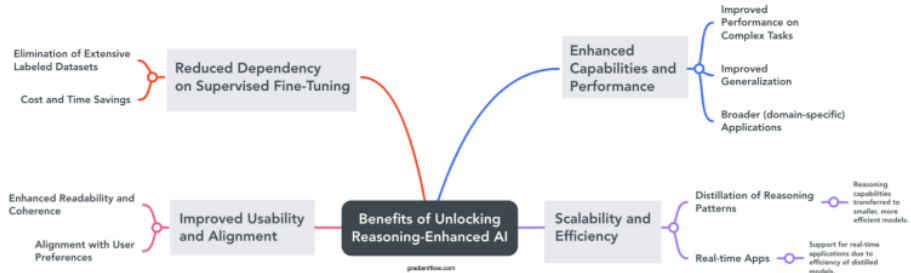
- Evaluation tasks too similar to training can overstate improvements.

► Key Insight:

- Improving base model quality is more impactful than just imitating proprietary systems.



Reinforcement Learning for Reasoning Skill Refinement



Open-Source and Community Contribution



Open-source models (e.g., DeepSeek-R1) for fine-tuning and deployment.



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

`research@deepseek.com`

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.



► What is Reinforcement Learning for Reasoning Skill Refinement?

- A method to enhance reasoning skills in LLMs through reinforcement learning.
- Focuses on refining the model's ability to perform reasoning tasks.

► How does it work?

- Uses a reward signal to guide the model towards better reasoning performance.
- Involves training the model on a variety of reasoning tasks with feedback.



► Why is it important?

- Enhances the model's reasoning capabilities, making it more effective for complex tasks.
- Addresses the limitations of traditional training methods by focusing on skill refinement.

► Challenges

- Requires a well-defined reward signal to effectively guide the learning process.
- Balancing exploration and exploitation during training can be difficult.



Examples of Models Using Reinforcement Learning for Reasoning Skill Refinement

► Deepseek-R1

- Deepseek-R1 utilizes reinforcement learning from human feedback (RLHF) to enhance its reasoning skills.
- The model is trained on a variety of reasoning tasks, receiving feedback to iteratively refine its logical and step-by-step problem-solving abilities.
- *Example:* Deepseek-R1 can tackle complex reasoning questions by learning from feedback on its intermediate reasoning steps as well as final answers.

Reinforcement Learning for Reasoning Skill Refinement (cont.)

أكاديمية كاوست
KAUST ACADEMY



LMH
Lady Margaret Hall

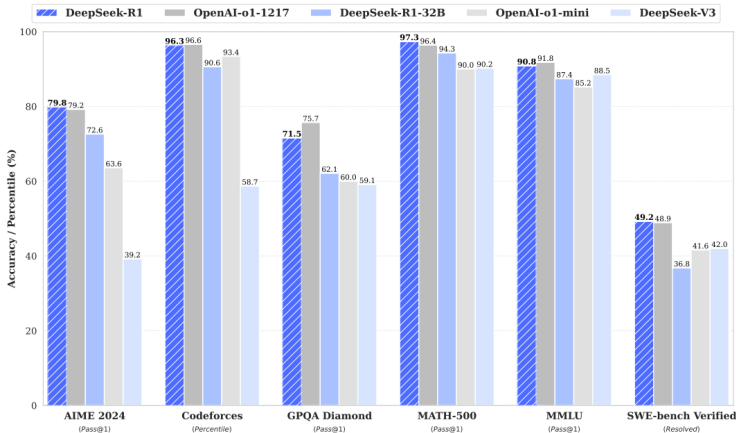


Figure 1 | Benchmark performance of DeepSeek-R1.

Source: [DeepSeek-AI \(2025\)](#)



► Llama with RLHF

- Llama models are fine-tuned using RLHF to refine their reasoning and decision-making skills.
- RLHF helps Llama models generate more accurate and contextually relevant responses in complex reasoning scenarios.
- *Example:* Llama can be trained to provide detailed justifications for its answers in logical deduction tasks.



► GPT-4 with RLHF

- GPT-4 uses RLHF extensively to align its outputs with human preferences, especially for tasks requiring nuanced reasoning.
- The model is exposed to a wide range of reasoning challenges and receives feedback to iteratively improve its performance.
- *Example:* GPT-4 can engage in Socratic dialogue, refining its reasoning through interactive feedback.



► Other Models

- Many recent LLMs (e.g., PaLM, Claude) incorporate RLHF or similar reinforcement learning techniques to enhance reasoning.
- These models benefit from iterative feedback and reward signals tailored to reasoning skill development.

Benchmark (Metric)	Claude-3.5- Sonnet-1022	GPT-4o 0513	DeepSeek V3	OpenAI o1-mini	OpenAI o1-1217	DeepSeek R1
Architecture	-	-	MoE	-	-	MoE
# Activated Params	-	-	37B	-	-	37B
# Total Params	-	-	671B	-	-	671B
English	MMLU (Pass@1)	88.3	87.2	88.5	85.2	91.8
	MMLU-Redux (EM)	88.9	88.0	89.1	86.7	-
	MMLU-Pro (EM)	78.0	72.6	75.9	80.3	-
	DROP (3-shot F1)	88.3	83.7	91.6	83.9	90.2
	IF-Eval (Prompt Strict)	86.5	84.3	86.1	84.8	-
	GPQA Diamond (Pass@1)	65.0	49.9	59.1	60.0	75.7
	SimpleQA (Correct)	28.4	38.2	24.9	7.0	47.0
	FRAMES (Acc.)	72.5	80.5	73.3	76.9	-
	AlpacaEval2.0 (LC-winrate)	52.0	51.1	70.0	57.8	-
Code	ArenaHard (GPT-4-1106)	85.2	80.4	85.5	92.0	-
	LiveCodeBench (Pass@1-COT)	38.9	32.9	36.2	53.8	63.4
	Codeforces (Percentile)	20.3	23.6	58.7	93.4	96.6
	Codeforces (Rating)	717	759	1134	1820	2061
	SWE Verified (Resolved)	50.8	38.8	42.0	41.6	48.9
	Aider-Polyglot (Acc.)	45.3	16.0	49.6	32.9	61.7
Math	AIME 2024 (Pass@1)	16.0	9.3	39.2	63.6	79.2
	MATH-500 (Pass@1)	78.3	74.6	90.2	90.0	96.4
	CNMO 2024 (Pass@1)	13.1	10.8	43.2	67.6	-
Chinese	CLUEWSC (EM)	85.4	87.9	90.9	89.9	-
	C-Eval (EM)	76.7	76.0	86.5	68.9	-
	C-SimpleQA (Correct)	55.4	58.7	68.0	40.3	-

Table 4 | Comparison between DeepSeek-R1 and other representative models.

Source: [DeepSeek-AI \(2025\)](#)



Reinforcement Learning for Reasoning in Large Language Models with *One* Training Example

Yiping Wang^{1 † *} Qing Yang² Zhiyuan Zeng¹ Liliang Ren³ Liyuan Liu³
Baolin Peng³ Hao Cheng³ Xuehai He⁴ Kuan Wang⁵ Jianfeng Gao³
Weizhu Chen³ Shuohang Wang^{3 †} Simon Shaolei Du^{1 †} Yelong Shen^{3 †}

¹University of Washington ²University of Southern California ³Microsoft

⁴University of California, Santa Cruz ⁵Georgia Institute of Technology

Source: Wang et al. (2025)

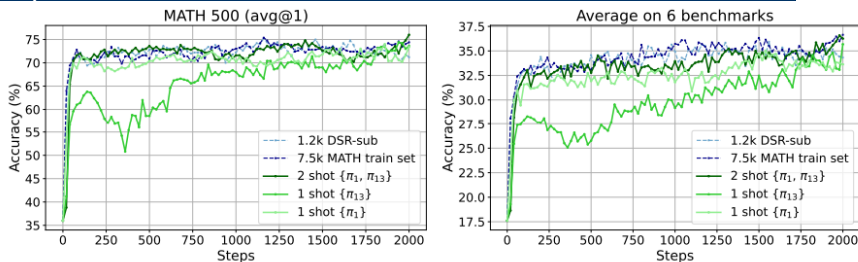


Figure 1: **RLVR with 1 example (green) can perform as well as using datasets with thousands of examples (blue).** Left/Right corresponds to MATH500/Average performance on 6 mathematical reasoning benchmarks (MATH500, AIME24, AMC23, Minerva Math, OlympiadBench, and AIME25). Base model is Qwen2.5-Math-1.5B. π_1 and π_{13} are examples defined by Eqn. 2 and detailed in Tab. 2, and they are from the 1.2k DeepScalerR subset (DSR-sub). Setup details are in Sec. 3.1. We find that RLVR with 1 example $\{\pi_{13}\}$ (35.7%) performs close to that with 1.2k DSR-sub (35.9%), and RLVR with 2 examples $\{\pi_1, \pi_{13}\}$ (36.6%) even performs better than RLVR with DSR-sub and as well as using 7.5k MATH train dataset (36.7%). Detailed results are in Fig. 6 in Appendix C.1.1. Additional results for non-mathematical reasoning tasks are in Tab. 1.

Source: Wang et al. (2025)

(cont.)

[illegible]

Model's response to training example 1 and a selected MATH500 problem. Green/Red are used for marking Correct/Wrong answers. Source: [Wang et al. \(2025\)](#)

Reinforcement Learning for Reasoning Skill Refinement (cont.)



RL Dataset	Dataset Size	MATH 500	AIME 2024	AMC 2023	Minerva Math	Olympiad-Bench	AIME 2025	Avg.
Qwen2.5-Math-7B [24] + GRPO								
NA	NA	51.0 ^{+0.0}	12.1 ^{+0.0}	35.3 ^{+0.0}	11.0 ^{+0.0}	18.2 ^{+0.0}	6.7 ^{+0.0}	22.4 ^{+0.0}
DSR-sub	1209	78.6 ^{+27.6}	25.8 ^{+13.7}	62.5 ^{+27.2}	33.8 ^{+22.8}	41.6 ^{+23.4}	14.6 ^{+7.9}	42.8 ^{+20.4}
$\{\pi_1\}$	1	79.2 ^{+28.2}	23.8 ^{+11.7}	60.3 ^{+25.0}	27.9 ^{+16.9}	39.1 ^{+20.9}	10.8 ^{+4.1}	40.2 ^{+17.8}
$\{\pi_1, \pi_{13}\}$	2	79.2 ^{+28.2}	21.7 ^{+9.6}	58.8 ^{+23.5}	35.3 ^{+24.3}	40.9 ^{+22.7}	12.1 ^{+5.4}	41.3 ^{+18.9}
$\{\pi_1, \pi_2, \pi_{13}, \pi_{1209}\}$	4	78.6 ^{+27.6}	22.5 ^{+10.4}	61.9 ^{+26.6}	36.0 ^{+25.0}	43.7 ^{+25.5}	12.1 ^{+5.4}	42.5 ^{+20.1}
Random	16	76.0 ^{+25.0}	22.1 ^{+10.0}	63.1 ^{+27.8}	31.6 ^{+20.6}	35.6 ^{+17.4}	12.9 ^{+6.2}	40.2 ^{+17.8}
$\{\pi_1, \dots, \pi_{16}\}$	16	77.8 ^{+26.8}	30.4 ^{+18.3}	62.2 ^{+26.9}	35.3 ^{+24.3}	39.9 ^{+21.7}	9.6 ^{+2.9}	42.5 ^{+20.1}
Llama-3.2-3B-Instruct [26] + GRPO								
NA	NA	40.8 ^{+0.0}	8.3 ^{+0.0}	25.3 ^{+0.0}	15.8 ^{+0.0}	13.2 ^{+0.0}	1.7 ^{+0.0}	17.5 ^{+0.0}
DSR-sub	1209	43.2 ^{+2.4}	11.2 ^{+2.9}	27.8 ^{+2.5}	19.5 ^{+3.7}	16.4 ^{+3.2}	0.8 ^{-0.9}	19.8 ^{+2.3}
$\{\pi_1\}$	1	45.8 ^{+5.0}	7.9 ^{-0.4}	25.3 ^{+0.0}	16.5 ^{+0.7}	17.0 ^{+3.8}	1.2 ^{-0.5}	19.0 ^{+1.5}
$\{\pi_1, \pi_{13}\}$	2	49.4 ^{+8.6}	7.1 ^{-1.2}	31.6 ^{+6.3}	18.4 ^{+2.6}	19.1 ^{+5.9}	0.4 ^{-1.3}	21.0 ^{+3.5}
$\{\pi_1, \pi_2, \pi_{13}, \pi_{1209}\}$	4	46.4 ^{+5.6}	6.2 ^{-2.1}	29.1 ^{+3.8}	21.0 ^{+5.2}	15.1 ^{+1.9}	1.2 ^{-0.5}	19.8 ^{+2.3}
Qwen2.5-Math-1.5B [24] + PPO								
NA	NA	36.0 ^{+0.0}	6.7 ^{+0.0}	28.1 ^{+0.0}	8.1 ^{+0.0}	22.2 ^{+0.0}	4.6 ^{+0.0}	17.6 ^{+0.0}
DSR-sub	1209	72.8 ^{+36.8}	19.2 ^{+12.5}	48.1 ^{+20.0}	27.9 ^{+19.8}	35.0 ^{+12.8}	9.6 ^{+5.0}	35.4 ^{+17.8}
$\{\pi_1\}$	1	72.4 ^{+36.4}	11.7 ^{+5.0}	51.6 ^{+23.5}	26.8 ^{+18.7}	33.3 ^{+11.1}	7.1 ^{+2.5}	33.8 ^{+16.2}
DeepSeek-R1-Distill-Qwen-1.5B [2] + GRPO								
NA	NA	71.0 ^{+0.0}	20.0 ^{+0.0}	60.9 ^{+0.0}	24.3 ^{+0.0}	30.2 ^{+0.0}	17.9 ^{+0.0}	37.4 ^{+0.0}
DSR-sub	1209	83.4 ^{+12.4}	27.5 ^{+7.5}	80.6 ^{+19.7}	32.0 ^{+7.7}	46.5 ^{+16.3}	24.6 ^{+6.7}	49.1 ^{+11.7}
$\{\pi_1\}$	1	78.0 ^{+7.0}	25.8 ^{+5.8}	71.6 ^{+10.7}	31.2 ^{+6.9}	39.3 ^{+9.1}	20.0 ^{+2.1}	44.3 ^{+6.9}
$\{\pi_1, \pi_2, \pi_{13}, \pi_{1209}\}$	4	78.8 ^{+7.8}	29.6 ^{+9.6}	75.6 ^{+14.7}	32.0 ^{+7.7}	40.9 ^{+10.7}	23.8 ^{+5.9}	46.8 ^{+9.4}
$\{\pi_1, \dots, \pi_{16}\}$	16	82.4 ^{+11.4}	30.4 ^{+10.4}	73.4 ^{+12.5}	34.2 ^{+9.9}	42.4 ^{+12.2}	24.6 ^{+6.7}	47.9 ^{+10.5}

1(few)-shot RLVR is viable for different models and RL algorithm. Source: Wang et al. (2025)



► Applications

- Can be applied to various reasoning tasks, such as logical reasoning, mathematical problem solving, and decision-making.
- Useful in scenarios where reasoning skills are critical for task success.

► Advanced Techniques

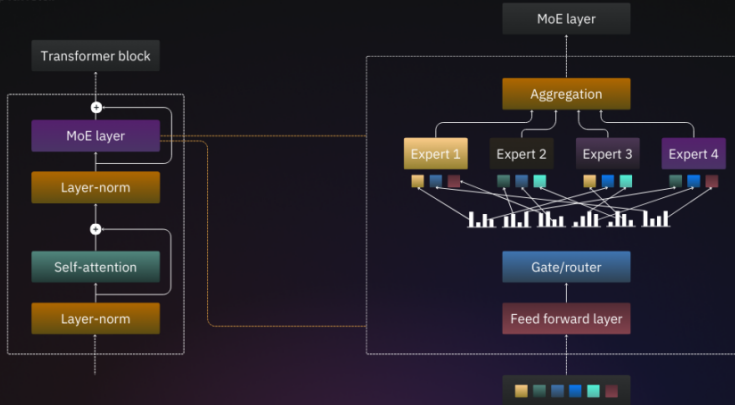
- Combining a reward model with a verifier model to ensure both performance and correctness.
- Rewarding attributes such as truthfulness, logical soundness, or helpfulness during training.

► Future Directions

- Further research is needed to improve the efficiency and effectiveness of reinforcement learning for reasoning skill refinement.
- Exploring different reward structures and training paradigms to enhance model performance.

What is a Mixture of Experts (MoE)?

neptune.ai



What is a Mixture of Experts (MoE)?

► Definition

- It combines multiple specialized models, called experts.
- Each expert focuses on a specific subset of data or task.
- This specialization enables efficient and effective learning.

► How does it work?

- MoE uses a gating mechanism.
- The gate selects which expert(s) to use for a given input.
- The model adapts its behavior dynamically based on the input.
- It leverages the strengths of different experts.

► Why is it important?

- MoE models handle large-scale problems efficiently.
- They distribute workload across multiple experts.
- Combining expertise of different models improves performance on complex tasks.

► Challenges

- Training MoE models is computationally expensive.
- Multiple experts need to be trained.
- Designing effective gating mechanisms is difficult.
- Ensuring balanced expert utilization is challenging.

Examples of Mixture of Experts Models

► Switch Transformer

- Uses a gating mechanism to select one or a few experts per input.
- Activates only a small subset of experts at a time.
- Enables large-scale models without excessive computational costs.
- *Example:*
 - Handles language modeling and translation.
 - Leverages specialized experts for different aspects of the task.

What is a Mixture of Experts (MoE)? (cont.)

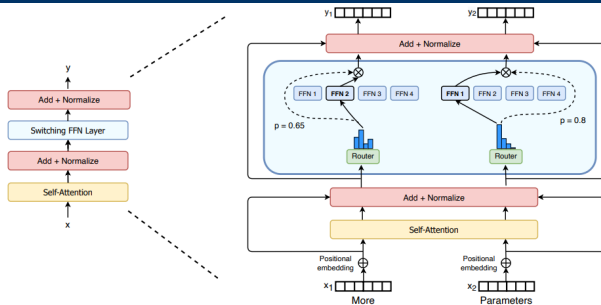


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = “More” and x_2 = “Parameters” below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Source: Fedus et al. (2021)

► GShard

- Scales up the number of experts for very large datasets and models.
- Distributes computation across multiple devices.
- Enables efficient training of large models.
- *Example:*
 - Used for image classification and NLP tasks.
 - Utilizes many experts to process different data parts.

What is a Mixture of Experts (MoE)? (cont.)

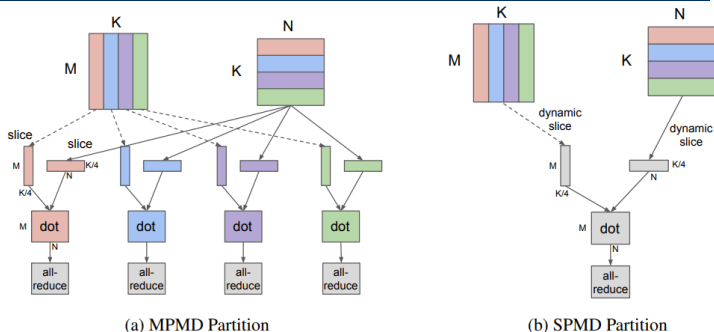


Figure 2: Comparison between MPMD and our proposed SPMD partitioning of a Dot operator ($[M, K] \times [K, N] = [M, N]$) across 4 devices. In this example, both operands are partitioned along the contracting dimension K , where each device computes the local result and globally combines with an AllReduce. MPMD partitioning generates separate operators for each device, limiting its scalability, whereas SPMD partitioning generates one program to run on all devices. Note that the compilation time with our SPMD partitioning is not-dependent of the number of devices being used.

Source: [Lepikhin et al. \(2020\)](#)

What is a Mixture of Experts (MoE) (MoE)? (cont.)

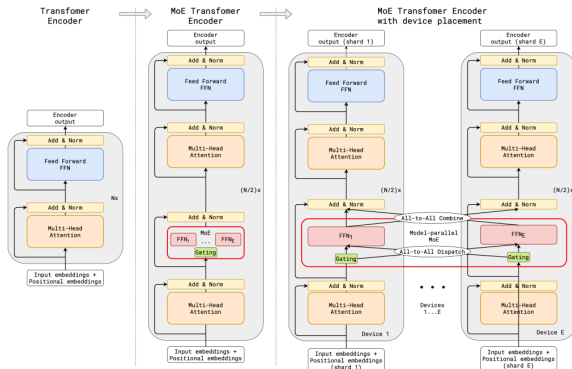


Figure 3: Illustration of scaling of Transformer Encoder with MoE Layers. The MoE layer replaces the every other Transformer feed-forward layer. Decoder modification is similar. (a) The encoder of a standard Transformer model is a stack of self-attention and feed forward layers interleaved with residual connections and layer normalization. (b) By replacing every other feed forward layer with a MoE layer, we get the model structure of the MoE Transformer Encoder. (c) When scaling to multiple devices, the MoE layer is sharded across devices, while all other layers are replicated.

Source: [Lepikhin et al. \(2020\)](#)

► MoE in BERT

- BERT can be extended with MoE layers.
- MoE layers enhance performance on specific tasks.
- Allows BERT to dynamically select experts based on input.
- Improves ability to handle diverse tasks.
- *Example:*
 - MoE-enhanced BERT performs better on sentiment analysis and question answering.
 - Uses specialized experts for different input types.

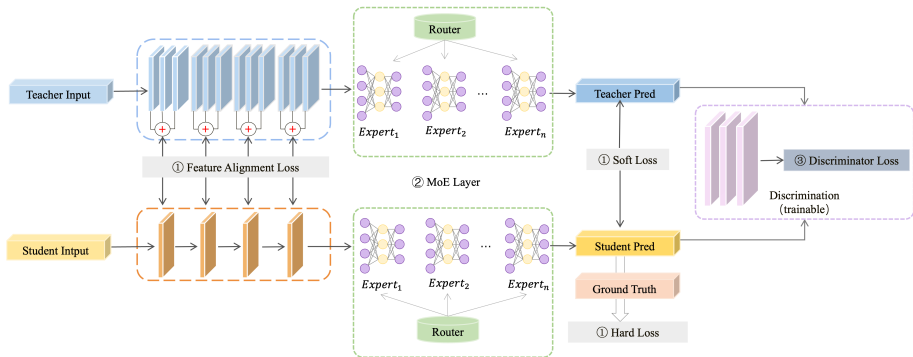
► MicroBERT: Distilling MoE-Based Knowledge

- Distills knowledge from MoE-enhanced BERT into a smaller model.
- Maintains high performance with reduced parameters.
- Uses knowledge distillation techniques.

What is a Mixture of Experts (MoE)? (cont.)

- Targets resource-constrained environments.
- *Key Points:*
 - ▶ Teacher: MoE-BERT; Student: MicroBERT.
 - ▶ Transfers both general and expert-specific knowledge.
 - ▶ Achieves competitive accuracy on NLP benchmarks.
 - ▶ Reduces inference time and memory usage.
 - ▶ Suitable for deployment on edge devices.

What is a Mixture of Experts (MoE)? (cont.)



Source: Zheng et al. (2024)

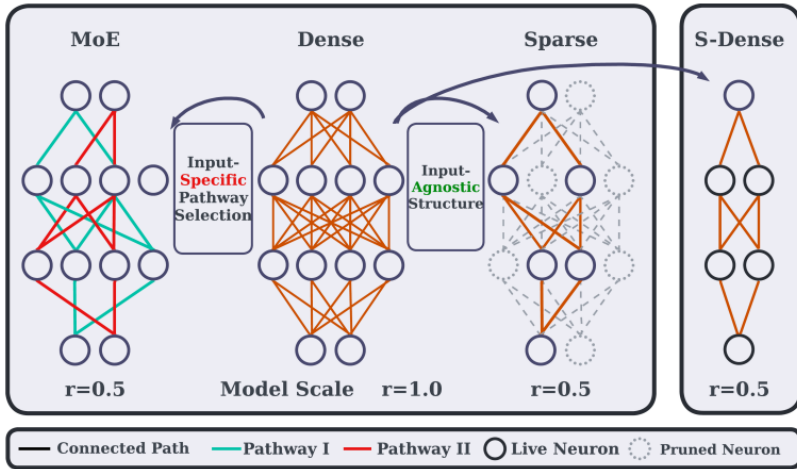
► Other MoE Models

- Recent models like T5 and GPT-3 use MoE architectures.
- MoE enables dynamic expert selection based on input.
- Improves learning efficiency and effectiveness.



Sparse vs. Dense Models: **Trade-offs**

Motivations



Source: Zhang et al. (2023)

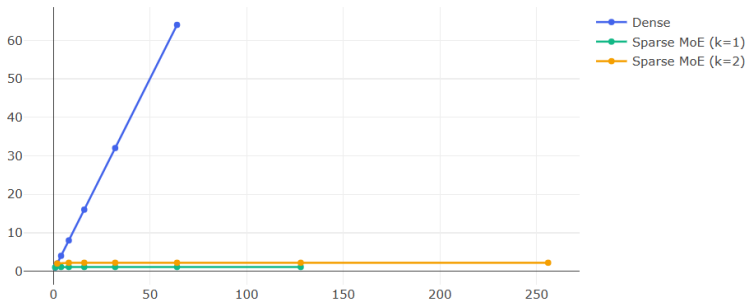
► Dense Models:

- Use a large number of parameters.
- All parameters participate for every input.
- Compute cost is predictable and uniform.
- Training is straightforward—no need for routing or selection.
- Require more computational resources.
- **Examples:**
 - Standard Transformer models (e.g., BERT, GPT).
 - Fully-connected neural networks.

► Sparse Mixture-of-Experts (MoE) Models:

- Only a subset (top- k) of experts are activated per input.
- Enables scaling to billions of parameters without increasing per-token compute.
- Requires a routing mechanism to select experts.
- Focus on specific features or patterns in the data.
- Efficient for large datasets with sparse representations.
- **Examples:**
 - Switch Transformer (Google).
 - GLaM (Google).
 - M6-T (Alibaba).

Sparse vs. Dense Models: Trade-offs Motivations (con



Comparison showing how FLOPs per token scale with total model parameters. Dense models exhibit coupled growth, while sparse MoE models allow parameter scaling with nearly constant FLOPs per token (assuming fixed k). Note the slight increase for MoE accounts for gating overhead and assumes experts scale parameters. Axes represent relative scale.

Source: [Contrasting Dense vs. Sparse Activation](#)



► Trade-offs:

- **Dense:** Simpler, stable, but limited by compute and memory.
- **Sparse:** Higher capacity, efficient compute, but adds routing overhead.
- Sparse models may face instability and load balancing issues.
- Choice depends on scale, task complexity, and infrastructure.

Gating Mechanisms – **How Experts Are Selected**

► Gating Mechanism:

- A crucial component in Mixture of Experts (MoE) models determines which expert(s) to activate for a given input.
- Uses a learned function to compute a score for each expert.
- The scores are used to select the top- k experts.
- (A small neural network that selects the top- k experts to use for a given input.)



► How it works:

- Input is passed through a gating network.
- The gating network outputs a probability distribution over experts.
- Only the experts with the highest probabilities are activated.
- This allows the model to focus on relevant experts for each input.



- **MoE Formulation**

- ▶ Input token: x
- ▶ Gating network G calculates expert distribution.

- **Gating Network Calculation**

- ▶ Linear transformation: $\text{logits} = W_g x$
- ▶ Softmax activation: $P = \text{softmax}(\text{logits})$
- ▶ W_g : trainable weight matrix
- ▶ P : probability vector for each expert

- **Dense MoE Output**

- ▶ Output $y = \sum_{i=1}^N P_i E_i(x)$
- ▶ Combines all experts with weights P_i .

- **Sparse MoE Goal**

- ▶ Focus on computational efficiency.
- ▶ Only activate a small subset of experts per token.
- ▶ Requires a mechanism to select experts based on P .

- **Prevalent Strategy**

- ▶ For sparse MoEs, selects a subset of experts.
- ▶ Gating network chooses k experts with highest probabilities.
- ▶ Typically, k is small (e.g., 1 or 2).

- **Process Overview**

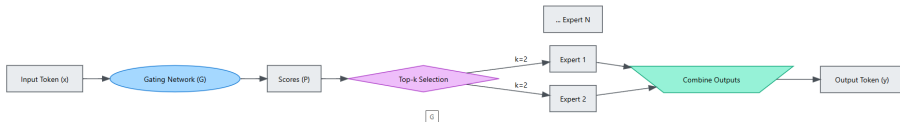
- ▶ **Compute Scores:** Calculate gating logits or probabilities (P).
- ▶ **Select Top-k:** Identify indices $I = \{i_1, i_2, \dots, i_k\}$ for top k values in P .
- ▶ **Re-normalize Scores (Optional):**
- ▶ P_{sel} : sub-vector of P for selected indices I .
- ▶ Re-normalized weights P' :

$$P'_j = \frac{\exp(\text{logits}_{i_j})}{\sum_{l=1}^k \exp(\text{logits}_{i_l})} \quad \text{for } j = 1 \dots k$$

- ▶ **Compute Weighted Output:**
- ▶ $y = \sum_{j=1}^k P'_j E_{i_j}(x)$
- ▶ Uses (re-normalized) probabilities as weights.

- **Choice of k**

- ▶ $k = 1$: Token routed to single expert.
- ▶ Maximizes sparsity.
- ▶ Limits knowledge combination.
- ▶ $k = 2$: Token uses two experts.
- ▶ Balances sparsity and representational capacity.
- ▶ Doubles computational cost vs. $k = 1$.
- ▶ k is a key hyperparameter affecting performance and cost.



Source: **Designing Effective Gating Networks**



► Load-Balanced + Entropy-Regularized Gating

- Ensures efficient and equal expert utilization.
- Prevents overloaded or underutilized experts.
- **Load-Balanced:**
 - Penalty in loss encourages even token distribution.
 - Avoids "expert collapse."
- **Entropy-Regularized:**
 - Promotes diverse expert selection.
 - Nudges model to use full range of experts.



► Examples in Practice

- **Example 1: Switch Transformer Gating**

- Primarily uses **Top-1 routing**.
- Incorporates **load balancing penalty**.
- Ensures experts are actively learning and distributed.

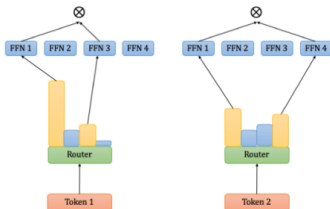
- **Example 2: MoE-LLaMA**

- Employs **Top-2 gating**.
- **Avoids token dropout**.
- Aims for smoother and consistent performance.

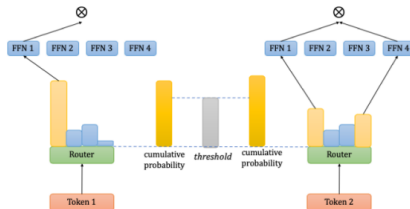
Routing in MoE

► Key Concepts:

- Assign tokens to experts per layer.
- Routing can be token-level or batch-level.
- Penalize overused experts (using load balancing loss).
- Efficient hardware utilization is key.



(a) Top-K routing



(b) Top-P routing

Figure 1: Comparison between Top-K routing mechanism and Top-P routing mechanism. (a) Each token selects fixed $K=2$ experts with Top-K routing probabilities. (b) In Top-P routing mechanism, each token selects experts with higher routing probabilities until the cumulative probability exceeds threshold.

Source: [Huandi et al. \(2024\)](#)

► Example 1: DeepSpeed-MoE

- Optimizes distributed routing and communication.
- Achieves linear scaling to hundreds of experts.
- Focuses on efficient data movement across devices.

Proposed Hierarchical AlltoAll Design

Complexity: $O(G+p/G)$ vs. $O(p)$ for basic alltoall
Legend: # GPUs (p) = 4, # GPUs/node (G) = 2, $p/G=2$

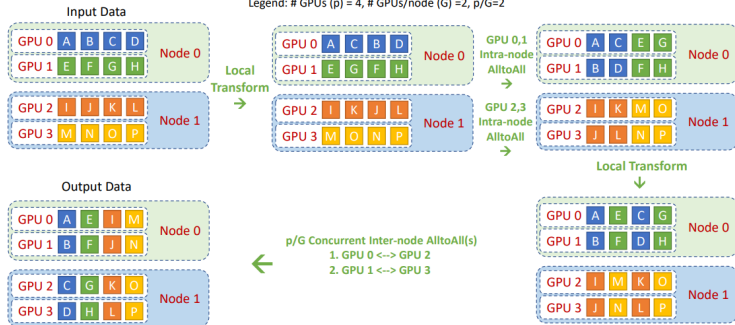


Figure 8: Illustration of the proposed hierarchical all-to-all design.

Source: DeepSpeed-MoE, Rajbhandari et al. (2022)

Routing in MoE (cont.)

Parallelism-coordinated AlltoAll optimization

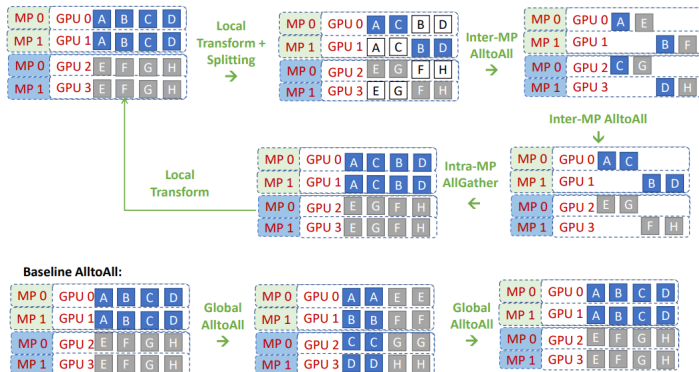


Figure 9: Illustration of the parallelism coordinated communication.

Source: DeepSpeed-MoE, Rajbhandari et al. (2022)

► Example 2: Tutel by Microsoft

- Uses custom **CUDA kernels**.
- For highly efficient token routing in MoE.
- Reduces overhead and speeds up computation on GPUs.

Routing in MoE (cont.)

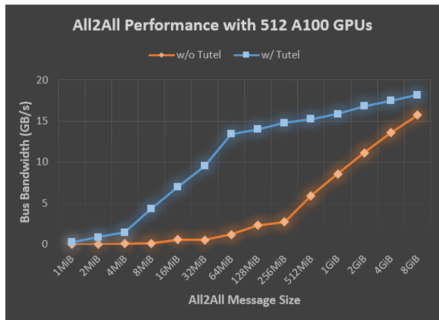
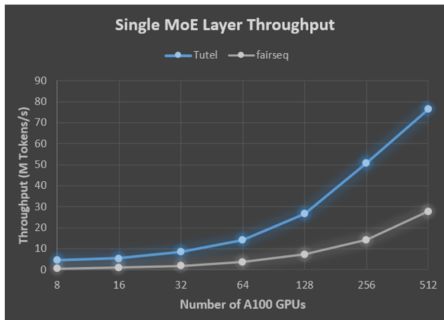


Figure 1 (left): Compared to fairseq, for a single MoE layer, Tutel achieves an 8.49x speedup on an NDm A100 v4 node with 8 GPUs and a 2.75x speedup on 64 NDm A100 v4 nodes with 512 A100 GPUs. The detailed setting is as follows: batch_size = 32, sequence_length = 1,024, Top_K = 2, model_dim = 2,048, and hidden_size = 2,048. Figure 2 (right): The all-to-all bandwidth for different message sizes with 64 NDm A100 v4 nodes (512 A100 GPUs) before and after applying Tutel. Tutel achieves a 2.56x to 5.93x all-to-all speedup with 512 A100 GPUs for message sizes hundreds of MiB large.

Source: Tutel (2021)



Benefits of MoE

► Scalability:

- Can scale to billions of parameters.
- Only a small subset of experts is activated for each input.
- Allows for larger models without proportional increases in compute.

► Efficient Computation:

- Activates only 10–20% of total parameters per token.
- Reduces computational cost per token.
- Sparse activation means fewer parameters are active at any time.
- Enables training on larger datasets with limited resources.
- Leads to lower latency and inference cost.
- Allows for models with vastly more parameters.

► Flexibility:

- Different experts can specialize in different tasks or data distributions.
- Allows for more nuanced model behavior and adaptability.
- Can be tailored to specific applications or domains.

► Specialization:

- Experts can focus on specific domains (e.g., math, code, biology, logic).
- Improves accuracy and performance in specialized tasks.

► Example 1: MoE with Domain Experts

- A math expert becomes active only on arithmetic inputs.
- Improves accuracy specifically for math problems.
- Outperforms generalist models on these tasks.

► Example 2: MORE (Mixture of Reasoning Experts)

- Explicitly uses experts for types of reasoning (e.g., multihop, factual, analogical).
- Boosts performance and provides greater interpretability.
- Allows understanding which expert handles which reasoning step.

► Performance:

- Often achieves state-of-the-art results on various benchmarks.
- Combines the strengths of multiple models without the overhead of training them separately.
- Can outperform dense models with fewer active parameters.



Limitations of LRMs and MoE Models

► For LRMs (Large Language Models):

- **Verbose outputs:** CoT (Chain-of-Thought) and ToT (Tree-of-Thought) methods generate long outputs.
- May confuse end-users due to extensive reasoning traces.
- **Prompt sensitivity:** Small changes in prompt wording lead to large changes in output.
- Requires careful prompt engineering.
- **Computation overhead:** Especially with self-consistency sampling or ToT trees.
- Increases inference cost and time.
- **Example:** CoT prompting works well on GSM8K (a math dataset), but fails without careful prompt tuning.
- Shows sensitivity to specific prompt design.

► For MoEs (Mixture of Experts):

- **Training instability:** Some experts may be underused, leading to "expert collapse."
- This is often addressed with load balancing.
- **Hard to debug:** Expert specialization may not always align with human-interpretable domains.
- Makes understanding model behavior challenging.
- **Distributed training cost:** Load balancing across GPUs adds significant engineering overhead.
- Requires complex distributed systems.

- **Example 1: GShard training instability**
 - ▶ Some experts never activate after several epochs of training.
 - ▶ Requires manual intervention or advanced balancing techniques.
- **Example 2: Switch Transformer**
 - ▶ Needed custom gradient balancing.
 - ▶ To prevent expert overload or dropout during training.

References & Further Reading

- ▶ Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. [arXiv:2201.11903](#).
- ▶ Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. [arXiv:2203.11171](#).
- ▶ DeepSeek-AI et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. [arXiv:2501.12948](#).
- ▶ Financial Times. (2025). *Transcript: The story behind DeepSeek's breakthrough*. [ft.com](#).
- ▶ Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. [arXiv:1701.06538](#).

- ▶ Shen, S., Hou, L., Zhou, Y., Du, N., Longpre, S., Wei, J., Chung, H. W., Zoph, B., Fedus, W., Chen, X., et al. (2023). *Mixture-of-Experts Meets Instruction Tuning: A Winning Combination for Large Language Models*. [arXiv:2305.14705](https://arxiv.org/abs/2305.14705).
- ▶ NVIDIA Developer Blog. (2023). *Applying Mixture of Experts in LLM Architectures*. developer.nvidia.com.
- ▶ Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., & Huang, J. (2024). *A Survey on Mixture of Experts in Large Language Models*. [arXiv:2407.06204](https://arxiv.org/abs/2407.06204).
- ▶ DataCamp. (2024). *What Is Mixture of Experts (MoE)? How It Works, Use Cases & More*. datacamp.com.
- ▶ Dai, D., et al. (2023). *Getting MoRE out of Mixture of Language Model Reasoning Experts*. In *Findings of EMNLP 2023*. [ACL Anthology](https://aclanthology.org/).

Credits

Dr. Prashant Aparajeya

Computer Vision Scientist — Director(AISimply Ltd)

p.aparajeya@aisimply.uk

This project benefited from external collaboration, and we acknowledge their contribution with gratitude.